



LISTA DE EXERCÍCIOS - SISTEMAS INTELIGENTES APLICADOS

Lista de Exercícios 8

Redes Neurais MLP

DOCENTE: Thiago França Naves

DATA: 30/09/2026

ALUNO: Mayara Rodrigues Pereira e Leonardo José Reis Pinto

A tabela-verdade e as regiões da questão 1 e as simulações das questões 2 e 3 foram feitas no script `solucao.py` (Apêndice A).

Questão 1

A figura abaixo mostra uma rede neural com um único neurônio escondido. Mostre que essa rede resolve o problema do XOR construindo (a) regiões de decisão e (b) uma tabela-verdade para a rede.

Resposta. Lendo os pesos da figura, o neurônio escondido recebe x_1 e x_2 com peso 1 e bias $-1,5$. O neurônio de saída recebe x_1 e x_2 diretamente, com peso 1, a saída do escondido com peso -2 , e tem bias $-0,5$. Usando o degrau (saída 1 quando a soma é maior ou igual a zero):

$$h = \text{degrau}(x_1 + x_2 - 1,5)$$

$$y = \text{degrau}(x_1 + x_2 - 2h - 0,5)$$

(b) Tabela-verdade.

x_1	x_2	net_h	h	net_y	y	XOR
0	0	$-1,5$	0	$-0,5$	0	0
0	1	$-0,5$	0	$0,5$	1	1
1	0	$-0,5$	0	$0,5$	1	1
1	1	$0,5$	1	$-0,5$	0	0

A coluna y é igual à do XOR nas quatro linhas.

(a) **Regiões de decisão.** O neurônio escondido sozinho é uma porta AND: só dispara acima da reta $x_1 + x_2 = 1,5$, ou seja, só com a entrada 11. A saída, quando $h = 0$, dispara acima da reta $x_1 + x_2 = 0,5$, que funciona como uma porta OR. Quando $h = 1$, o peso -2 desloca a reta da saída para $x_1 + x_2 = 2,5$, fora do quadrado, e a saída vai para 0. O resultado é uma faixa entre as duas retas paralelas, que contém 01 e 10 e deixa 00 de um lado e 11 do outro. É o “OR, mas não AND”, que é o XOR.

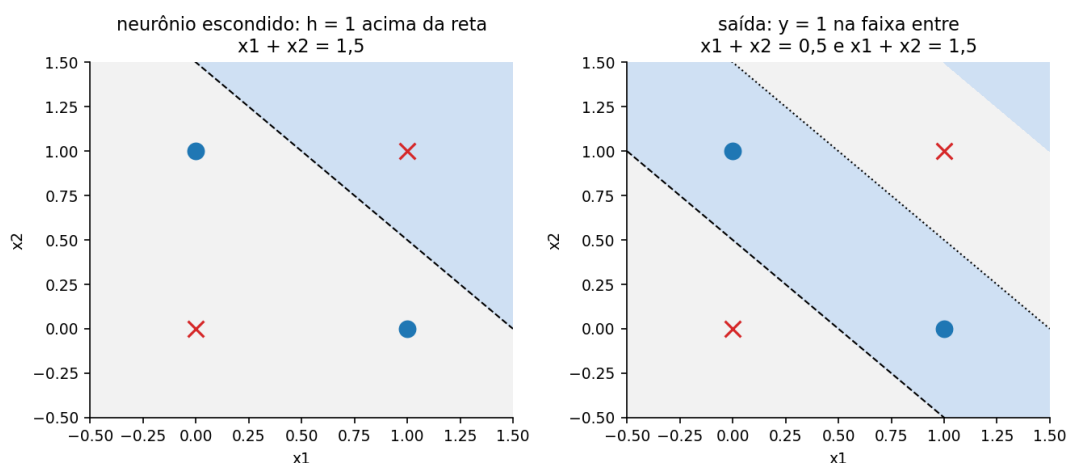


Figura 1. Regiões em que o neurônio escondido (esquerda) e a saída (direita) valem 1, em azul. Círculos são XOR = 1 e X são XOR = 0.

Um perceptron sozinho não resolve o XOR porque só consegue uma reta. Aqui o neurônio escondido cria um atributo novo, h (“as duas entradas estão ligadas”), e com ele a saída consegue separar as classes. A ligação direta das entradas com a saída permite fazer isso com um único neurônio escondido.

Questão 2

A constante momentum α tem-se normalmente atribuído um valor positivo na faixa $0 \leq \alpha < 1$. Investigue a diferença que ocorreria no comportamento da equação abaixo com respeito ao tempo t se α for atribuído um valor negativo na faixa $-1 < \alpha \leq 0$.

$$\Delta w_{ji}(n) = -\eta \sum_{t=0}^n \alpha^{n-t} \frac{\partial \xi(t)}{\partial w_{ji}(t)}$$

Resposta. A equação diz que a correção do peso na iteração n é uma soma de todos os gradientes anteriores, e o gradiente de t iterações atrás entra com peso α^{n-t} . Com $\alpha = 0,5$, os pesos dos gradientes, do mais recente para o mais antigo, são 1; 0,5; 0,25; 0,125... Com $\alpha = -0,5$ são 1; -0,5; 0,25; -0,125..., com o sinal alternando.

Com α positivo, a correção é uma média exponencial dos gradientes. Se o gradiente mantém o sinal por várias iterações, as parcelas se somam e o passo cresce até $1/(1 - \alpha)$ vezes o normal, o que acelera a descida em regiões planas. Se o gradiente troca de sinal a cada iteração (oscilação num vale estreito), as parcelas se cancelam e a oscilação é amortecida.

Com α negativo acontece o contrário nas duas situações:

- com gradiente de sinal constante, cada gradiente antigo é descontado do novo, e o passo acumulado cai para $1/(1 - \alpha) = 1/(1 + |\alpha|)$ vezes o normal, por exemplo $2/3$ com $\alpha = -0,5$. O treino fica mais lento, como se tivesse um freio;
- com gradiente que alterna de sinal, o produto do sinal alternado do gradiente com o sinal alternado de α^{n-t} deixa todas as parcelas com o mesmo sinal. As oscilações são reforçadas em vez de amortecidas, e a faixa de taxas de aprendizado em que o treino converge fica menor.

Com $\alpha = 0$ a soma tem uma parcela só e volta a ser o backpropagation comum. O α negativo não traz vantagem, e é por isso que se usa $0 \leq \alpha < 1$. A simulação da questão 3 mostra isso: com

$\eta = 0,05$ o treino sem momento leva 94 épocas, com $\alpha = -0,5$ leva 147, e com $\eta = 0,9$ qualquer α negativo faz o peso divergir.

Questão 3

Considere o exemplo simples de uma rede envolvendo um único peso, para a qual a função de custo é $\xi(w) = k_1(w - w_0)^2 + k_2$, onde w_0 , k_1 e k_2 são constantes. Um algoritmo back-propagation com momentum é usado para minimizar $\xi(w)$. Explore a forma na qual a inclusão da constante momentum α influencia o processo de aprendizado, com referência ao número de épocas requeridas para a convergência versus α .

Resposta. O gradiente é $\partial\xi/\partial w = 2k_1(w - w_0)$, e a regra com momento é

$$\Delta w(n) = -\eta 2k_1(w(n) - w_0) + \alpha \Delta w(n-1), \quad w(n+1) = w(n) + \Delta w(n).$$

O k_2 não aparece, porque só desloca o custo e não muda o gradiente. Usamos $k_1 = 1$, $w_0 = 2$, começamos em $w = 0$ e contamos as épocas até $|w - w_0| < 10^{-4}$, variando α de $-0,9$ a $0,9$ para quatro valores de η .

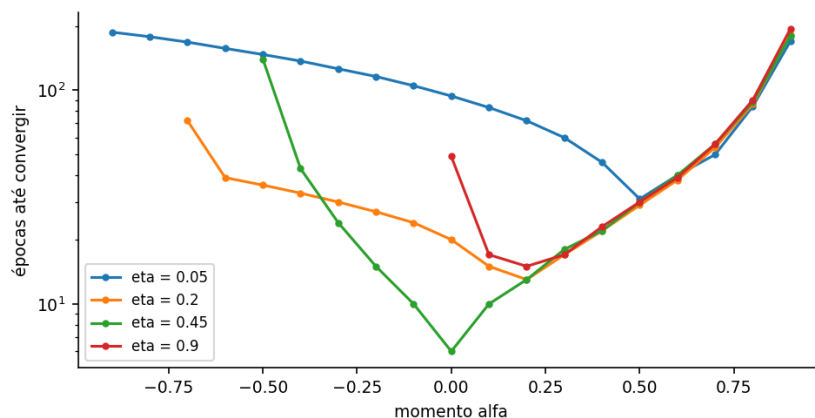


Figura 2. Épocas até convergir em função de α (escala log). Pontos que faltam divergiram.

Tabela 1. Épocas até convergir. O traço indica divergência.

η	$\alpha = -0,5$	$-0,2$	0	0,2	0,5	0,8	melhor
0,05	147	116	94	72	31	84	31 ($\alpha = 0,5$)
0,2	36	27	20	13	29	86	13 ($\alpha = 0,2$)
0,45	140	15	6	13	30	88	6 ($\alpha = 0$)
0,9	—	—	49	15	30	90	15 ($\alpha = 0,2$)

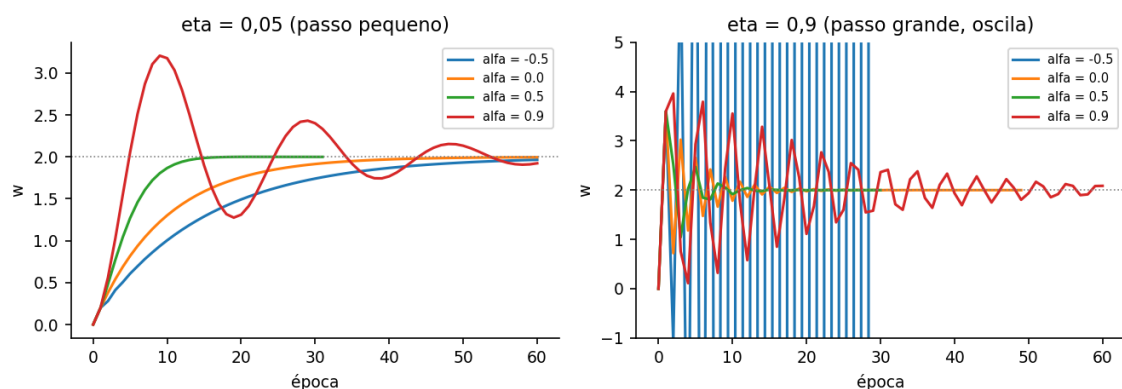


Figura 3. Valor de w ao longo das épocas.

O efeito do momento depende do tamanho do passo:

- Com passo pequeno ($\eta = 0,05$) o peso anda devagar e sempre na mesma direção. O momento acumula esses passos e acelera muito: de 94 épocas sem momento para 31 com $\alpha = 0,5$.
- Com o passo quase ideal ($\eta = 0,45$, em que $2\eta k_1 = 0,9$ e a descida chega perto de w_0 em poucas épocas) o momento só atrapalha: o melhor é $\alpha = 0$, com 6 épocas.
- Com passo grande ($\eta = 0,9$) o peso passa do mínimo e oscila de um lado para o outro. Um momento pequeno amortece essa oscilação e reduz de 49 para 15 épocas.
- Com α perto de 1, o treino fica lento para qualquer η (cerca de 180 épocas com $\alpha = 0,9$), porque o peso ganha “inércia”, passa do mínimo e fica oscilando em volta dele por muito tempo, como mostra a curva de $\alpha = 0,9$.

Então o número de épocas em função de α tem forma de U: cai até um valor ótimo e depois volta a subir. O ótimo depende de η . Para esse custo quadrático dá para mostrar que ele fica perto de $\alpha^* = (1 - \sqrt{2\eta k_1})^2$. Com $\eta = 0,05$ isso dá 0,47, perto do 0,5 que encontramos. Com α negativo o treino é sempre mais lento que sem momento, e a condição de convergência, $0 < 2\eta k_1 < 2(1 + \alpha)$, fica mais apertada. Por isso $\eta = 0,9$ diverge com qualquer α negativo.

Questão 4

Quando se justifica utilizar num MLP duas camadas (uma escondida) ou três camadas (duas escondidas) e que tipos de funções podem ser implementadas em cada caso?

Resposta. Uma camada escondida é suficiente para aproximar qualquer função contínua com a precisão que se quiser, desde que tenha neurônios suficientes (teorema da aproximação universal). Em classificação, ela consegue formar qualquer região convexa, como a interseção dos semiplanos criados por cada neurônio, e também regiões mais complicadas se tiver muitos neurônios. É a primeira opção para a maioria dos problemas.

Duas camadas escondidas se justificam quando as regiões de decisão são não convexas ou formadas por várias partes separadas, ou quando a função tem descontinuidades. A primeira camada monta semiplanos, a segunda combina esses semiplanos em regiões convexas e a saída une essas regiões, o que permite qualquer formato. Duas camadas pequenas podem representar com menos neurônios o que uma única camada larga precisaria de muitos para fazer, mas são mais difíceis de treinar. A regra é começar com uma camada escondida e só acrescentar a segunda se a validação mostrar que falta capacidade.

Questão 5

Que taxa de aprendizado η você usaria para treinar um MLP quando os padrões de uma mesma classe são semelhantes a alguns padrões de outra classe? E se forem bastante diferentes?

Resposta. Se as classes se sobrepõem, a fronteira de decisão precisa ser ajustada com cuidado e cada exemplo ambíguo empurra os pesos para um lado diferente. Com η alto os pesos ficariam oscilando de um exemplo para o outro sem assentar. Usaríamos uma taxa **pequena** (por exemplo, entre 0,01 e 0,1), de preferência com momento e diminuindo ao longo do treino.

Se as classes são bem diferentes, qualquer fronteira razoável entre elas funciona e o erro cai de forma consistente. Dá para usar uma taxa **maior** (entre 0,1 e 0,5), que converge em menos épocas, sem risco de oscilar em volta de uma fronteira delicada.

Questão 6

O que são mínimos locais? Como sair deles?

Resposta. A função de erro de um MLP tem muitos vales. Um mínimo local é um ponto em que o erro é menor que em toda a vizinhança, mas maior que o do melhor ponto possível (o mínimo global). O gradiente ali é zero, e o gradiente descendente para de andar achando que chegou ao fim.

Algumas formas de escapar ou evitar:

- usar momento, para o peso passar por cima de vales pequenos com a velocidade que já tinha;
- treinar por exemplo ou em pequenos lotes (gradiente estocástico), porque o ruído das atualizações tira o peso de vales rasos;
- treinar várias vezes com pesos iniciais aleatórios diferentes e ficar com a melhor rede;
- usar uma taxa de aprendizado que começa alta e vai diminuindo;
- acrescentar neurônios, o que deixa a superfície de erro com mais caminhos de descida.

Questão 7

Por que o back-propagation é chamada de regra delta generalizada?

Resposta. A regra delta (de Widrow e Hoff) ajusta os pesos de uma única camada na direção que diminui o erro quadrático: $\Delta w_i = \eta (t - y) x_i$. Ela só funciona na saída, onde o erro $t - y$ é conhecido. O backpropagation faz a mesma conta em todas as camadas: na saída o termo de erro é $\delta = (t - y) f'(\text{net})$, e numa camada escondida, onde não existe saída desejada, ele é calculado a partir dos δ da camada seguinte, $\delta_j = f'(\text{net}_j) \sum_k \delta_k w_{kj}$, usando a regra da cadeia. A atualização continua sendo $\Delta w = \eta \delta x$. É a regra delta estendida para redes com camadas escondidas e funções de ativação não lineares, e daí o nome.

Questão 8

Por que se utiliza gradiente descendente para treinar MLP?

Resposta. Porque treinar é minimizar uma função de erro em relação a milhares de pesos, e não existe fórmula fechada para o mínimo quando há camadas com funções não lineares. O gradiente diz, para cada peso, em que direção o erro aumenta, e andar um pouco na direção contrária diminui o erro. O backpropagation calcula esse gradiente para todos os pesos de uma vez, com custo parecido com o de uma passada pela rede, o que torna o método viável mesmo em redes grandes. Por isso as funções de ativação precisam ser deriváveis, e a sigmoide substituiu o degrau do perceptron.

Questão 9

O que é overfitting?

Resposta. É quando o modelo aprende os exemplos de treino bem demais, incluindo o ruído e os detalhes que não se repetem em dados novos. O erro no treino continua caindo, mas o erro em dados que a rede não viu para de cair e começa a subir. Num MLP acontece com muitos neurônios para poucos exemplos ou com épocas demais. Para evitar: separar um conjunto de validação e parar o treino quando o erro nele começar a subir, reduzir o tamanho da rede, usar regularização (penalizar pesos grandes, dropout) e conseguir mais dados.

Apêndice A. Código

```
pip install numpy matplotlib
python solucao.py

# Lista 8 - Redes Neurais MLP
# Sistemas Inteligentes Aplicados - UTFPR
# Uso: python solucao.py

import os
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

PASTA = os.path.dirname(os.path.abspath(__file__))
SAIDA = os.path.join(PASTA, "saidas")
os.makedirs(SAIDA, exist_ok=True)
plt.rcParams.update({"font.size": 9, "axes.spines.top": False,
                    "axes.spines.right": False})

def degrau(v):
    return (np.asarray(v) >= 0).astype(int)

# questao 1: pesos da figura
def rede(x1, x2):
    h = degrau(x1 + x2 - 1.5)
    y = degrau(x1 + x2 - 2 * h - 0.5)
    return h, y

print("[1] tabela-verdade da rede")
print(" x1 x2 | net_h h | net_y y | XOR")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        h, y = rede(x1, x2)
        print(f" {x1} {x2} | {x1 + x2 - 1.5:5.1f} {h} | "
              f" {x1 + x2 - 2 * h - 0.5:5.1f} {y} | {x1 ^ x2}")

g = np.linspace(-0.5, 1.5, 401)
X1, X2 = np.meshgrid(g, g)
H, Y = rede(X1, X2)
fig, ax = plt.subplots(1, 2, figsize=(9, 4))
ax[0].contourf(X1, X2, H, levels=[-0.5, 0.5, 1.5],
               colors=["#f2f2f2", "#cfe0f3"])
ax[0].plot(g, 1.5 - g, "k--", lw=1)
ax[0].set_title("neurônio escondido: h = 1 acima da reta\nx1 + x2 = 1,5")
ax[1].contourf(X1, X2, Y, levels=[-0.5, 0.5, 1.5],
               colors=["#f2f2f2", "#cfe0f3"])
ax[1].plot(g, 0.5 - g, "k--", lw=1)
ax[1].plot(g, 1.5 - g, "k:", lw=1)
ax[1].set_title("saída: y = 1 na faixa entre\nx1 + x2 = 0,5 e x1 + x2 = 1,5")
for a in ax:
    for x1 in [0, 1]:
        for x2 in [0, 1]:
            cor = "tab:blue" if x1 ^ x2 else "tab:red"
            a.scatter(x1, x2, s=90, color=cor, marker="o" if x1 ^ x2 else "x",
                     zorder=3)
    a.set_xlim(-0.5, 1.5)
    a.set_ylim(-0.5, 1.5)
    a.set_xlabel("x1")
    a.set_ylabel("x2")
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q1_regioes.png"), dpi=150)
plt.close(fig)

# questoes 2 e 3: custo E(w) = k1 (w - w0)^2 + k2
K1, K2, W0 = 1.0, 0.5, 2.0
```

```

def descer(eta, alfa, w=0.0, tol=1e-4, max_epocas=2000):
    delta = 0.0
    caminho = [w]
    for epoca in range(1, max_epocas + 1):
        grad = 2 * K1 * (w - W0)
        # regra com momento: delta(n) = -eta grad + alfa delta(n-1)
        delta = -eta * grad + alfa * delta
        w += delta
        caminho.append(w)
        if abs(w - W0) < tol and abs(delta) < tol:
            return epoca, caminho
        if abs(w) > 1e6:
            return None, caminho
    return None, caminho

print("\n[2] peso dos gradientes antigos, alfa^(n-t)")
for alfa in [0.5, -0.5]:
    print(f"  alfa={alfa}: " + " ".join(f"{alfa ** k:+.3f}" for k in range(6)))

print("\n[3] épocas até |w - w0| < 1e-4")
alfas = np.round(np.arange(-0.9, 0.95, 0.1), 2) + 0.0
etas = [0.05, 0.2, 0.45, 0.9]
resultados = {}
for eta in etas:
    linha = []
    for a in alfas:
        ep, _ = descer(eta, a)
        linha.append(ep)
    resultados[eta] = linha
    melhor = min((e, a) for e, a in zip(linha, alfas) if e)
    txt = " ".join(f"{a:+.1f}:{e if e else '--'}" for a, e in zip(alfas, linha))
    print(f"  eta={eta}: {txt}")
    print(f"    menor número de épocas: {melhor[0]} com alfa={melhor[1]:+.1f}")

fig, ax = plt.subplots(figsize=(6.5, 3.4))
for eta in etas:
    ep = [e if e else np.nan for e in resultados[eta]]
    ax.plot(alfas, ep, "o-", ms=3, label=f"eta = {eta}")
ax.set_yscale("log")
ax.set_xlabel("momento alfa")
ax.set_ylabel("épocas até convergir")
ax.legend(fontsize=8)
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q3_epocas_alfa.png"), dpi=150)
plt.close(fig)

fig, ax = plt.subplots(1, 2, figsize=(9, 3.2))
for alfa in [-0.5, 0.0, 0.5, 0.9]:
    _, c = descer(0.05, alfa, max_epocas=60)
    ax[0].plot(c[:61], label=f"alfa = {alfa}")
    _, c = descer(0.9, alfa, max_epocas=60)
    ax[1].plot(c[:61], label=f"alfa = {alfa}")
ax[0].set_title("eta = 0,05 (passo pequeno)")
ax[1].set_title("eta = 0,9 (passo grande, oscila)")
for a in ax:
    a.axhline(W0, color="gray", lw=0.8, ls=":")
    a.set_xlabel("época")
    a.set_ylabel("w")
    a.legend(fontsize=7)
ax[1].set_ylim(-1, 5)
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q3_trajetorias.png"), dpi=150)
plt.close(fig)

```